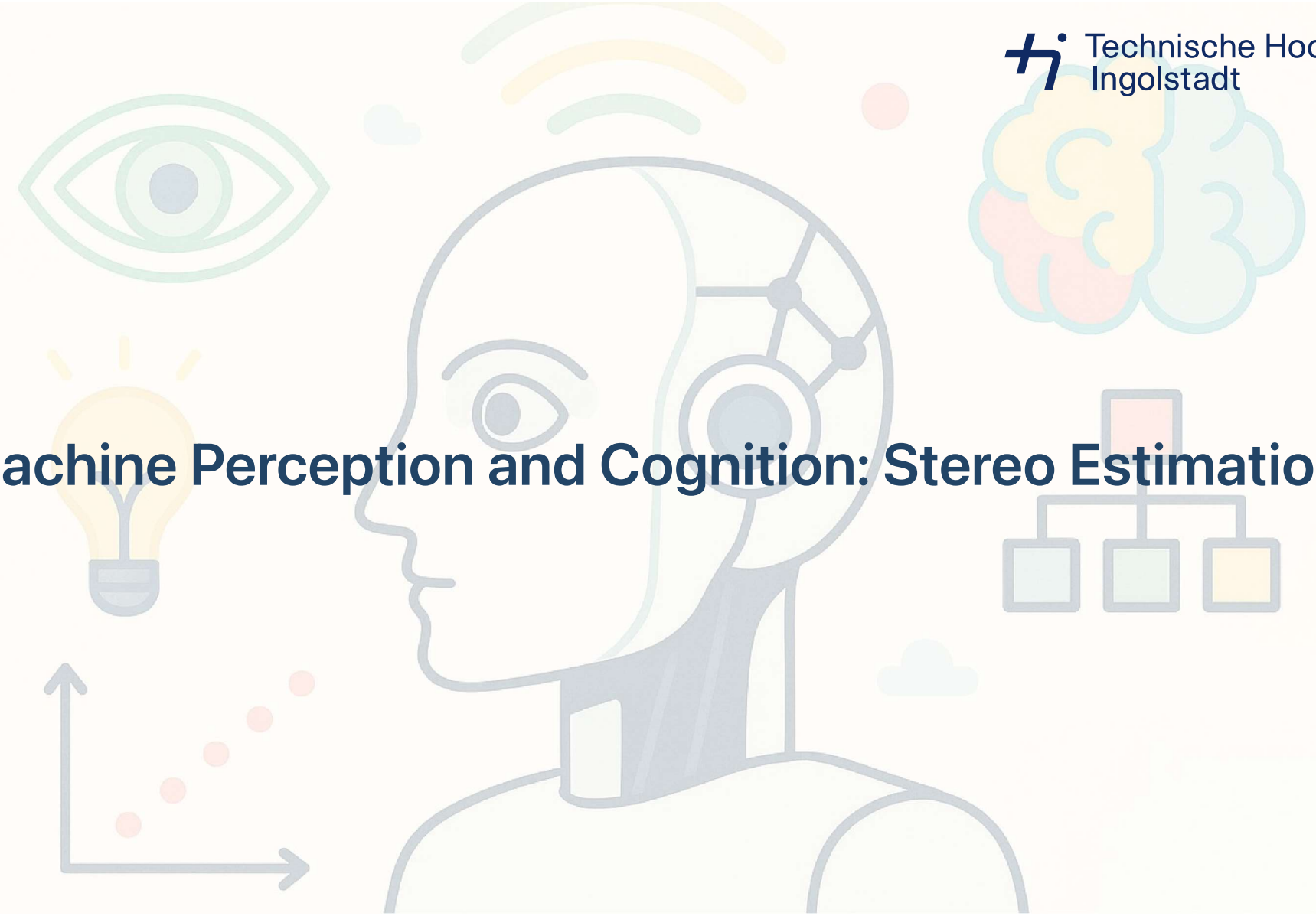
Technische Hochschule Ingolstadt



Machine Perception and Cognition: Stereo Estimation

Motivation: Stereo Estimation

Humans perceive depth effortlessly using **two eyes** — the small horizontal offset between the left and right eye provides a natural depth cue.

Stereo estimation (or stereo depth estimation) mimics this principle using two cameras:

- Given a **stereo pair** of images, estimate the **disparity** of corresponding pixels and convert it to **depth** where valid
- Output: typically a **dense disparity map** and, via calibration, a corresponding **depth map**

Motivation: Stereo Estimation

Applications:

- Autonomous driving (obstacle distance estimation)
- Robotics (grasping, navigation)
- 3D reconstruction (mapping, augmented reality)
- Industrial inspection (dimensional measurement)

Binocular Depth Perception

In human vision, the brain exploits **binocular disparity**: the same 3D point projects to slightly different positions on the left and right retina.

- The horizontal shift is larger for **near objects** and smaller for **far objects**
- The brain fuses both images to infer depth — this is called **stereopsis**

A stereo camera system is the engineering analogue: two rigidly mounted cameras with known geometry replace the two eyes.

Binocular Depth Perception

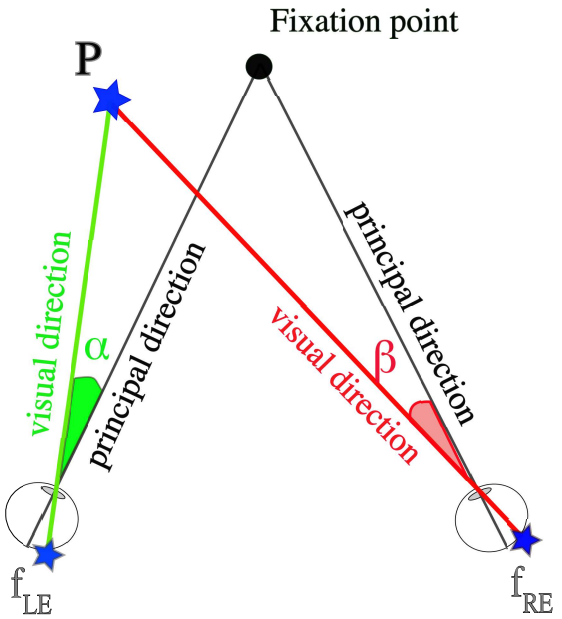


Image source: Jodi.Krol, CC BY-SA 4.0 (https://creativecommons.org/licenses/by-sa/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:En.hoofdrichting.svg

The Stereo Camera Setup

A typical stereo camera system consists of:

- **Left camera** C_L and **right camera** C_R
- After **rectification**, corresponding epipolar lines are horizontal and matching points lie on the same image row
- Separated by a **baseline** b — the horizontal distance between the camera centers

Key design parameters:

Parameter	Effect
Baseline b	Larger b → better depth at range
Focal length f	Larger f → finer disparity resolution
Image resolution	Higher resolution → finer detail

Pinhole Camera Model — Recap

The **pinhole camera** maps a 3D point $\mathbf{P} = (X, Y, Z)^\top$ in camera coordinates to a 2D image point $\mathbf{p} = (u, v)^\top$:

$$u = f_x \frac{X}{Z} + c_x, \qquad v = f_y \frac{Y}{Z} + c_y$$

In homogeneous coordinates:

$$\lambda \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \mathbf{K} \begin{pmatrix} X \\ Y \\ Z \end{pmatrix}, \qquad \mathbf{K} = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix}$$

where $\mathbf{K}$ is the **intrinsic matrix**, f_x, f_y are the focal lengths in pixels, and (c_x, c_y) is the **principal point**.

Coordinate Systems in a Stereo Rig

A stereo rig defines multiple coordinate frames:

- **World frame**: fixed reference frame in the scene
- **Left camera frame** $\{L\}$: origin at the left camera's optical center
- **Right camera frame** $\{R\}$: origin at the right camera's optical center
- **Image frame**: 2D pixel coordinates for each camera

Coordinate Systems in a Stereo Rig

The **extrinsic** transformation from the right to the left camera frame is described by a rotation $\mathbf{R}$ and translation $\mathbf{t}$:

$$\mathbf{P}_L = \mathbf{R}\mathbf{P}_R + \mathbf{t}$$

For a rectified horizontal stereo rig: $\mathbf{R} = \mathbf{I}$ and $\mathbf{t} = (b, 0, 0)^\top$, so equivalently $\mathbf{P}_R = \mathbf{P}_L - (b, 0, 0)^\top$. This is a change of coordinate frame: if the right camera center is displaced by $+b$ along the left camera's x-axis, then the same 3D point has an x-coordinate $X_R = X_L - b$ in the right camera frame.

Projection in a Stereo Camera Pair

For a 3D point $\mathbf{P} = (X, Y, Z)^\top$ expressed in the **left camera frame**, and assuming a rectified stereo pair with identical intrinsics and $f_x = f_y = f$, the projections onto the two image planes are:

Left image:

$$u_L = f\frac{X}{Z} + c_x, \qquad v_L = f\frac{Y}{Z} + c_y$$

Right image (with baseline b along X):

$$u_R = f\frac{X - b}{Z} + c_x, \qquad v_R = f\frac{Y}{Z} + c_y$$

Note: the v coordinates are **identical** after rectification — the matching problem becomes **1-dimensional** along the horizontal axis.

Epipolar Geometry

Epipolar geometry describes the geometric relationship between two views of the same scene.

Intuition: imagine shooting a ray from the left camera center through a pixel $\mathbf{p}_L$. The observed 3D point lies somewhere along that ray — it could be near or far. When that entire ray is projected into the right camera it traces a straight line: the **epipolar line**.

Key insight: the correct match $\mathbf{p}_R$ for $\mathbf{p}_L$ must lie on this epipolar line.

This reduces the **2D search** for correspondences to a **1D search** along a line, dramatically reducing ambiguity and computation.

Epipolar Geometry

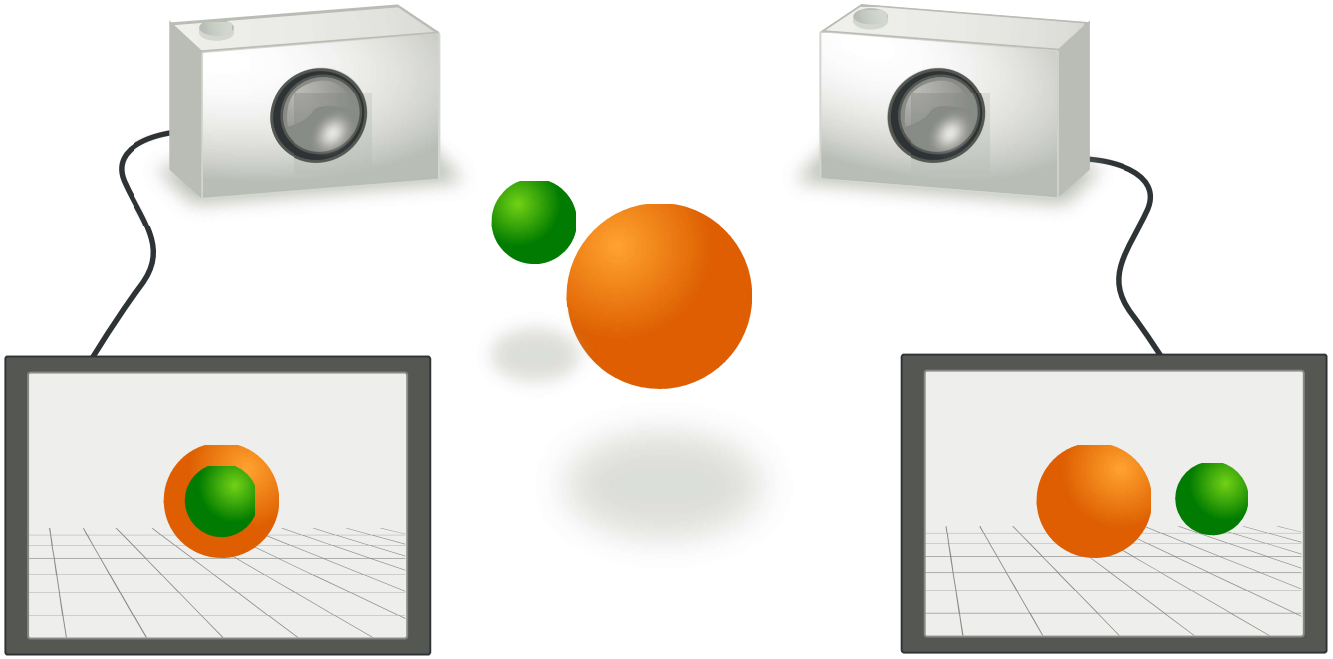


Image source: Arne Nordmann (norro), CC BY-SA 3.0 (https://creativecommons.org/licenses/by-sa/3.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Aufnahme_mit_zwei_Kameras.svg

The Epipole

The **epipole** $\mathbf{e}$ in one image is the projection of the other camera's optical center onto that image's plane:

- $\mathbf{e}_L$ = projection of C_R onto the left image
- $\mathbf{e}_R$ = projection of C_L onto the right image

The epipole is the **vanishing point** of all epipolar lines in that image.

For a **rectified stereo pair** with a horizontal baseline, the epipoles are at infinity — all epipolar lines are **horizontal** and **parallel**, which greatly simplifies the matching problem.

The Epipolar Plane and Epipolar Lines

For any 3D point $\mathbf{P}$, the three points $\mathbf{P}, C_L, C_R$ define the **epipolar plane**.

The epipolar plane intersects each image plane in a line — the **epipolar line**:

- Right epipolar line l_R: the constrained search region for $\mathbf{p}_R$ given $\mathbf{p}_L$
- Left epipolar line l_L: the constrained search region for $\mathbf{p}_L$ given $\mathbf{p}_R$

The epipolar constraint: $\mathbf{p}_R$ lies on the epipolar line l_R induced by $\mathbf{p}_L$, and vice versa.

Stereo Rectification

Rectification is an image transformation applied to both camera images so that:

- Corresponding points in the left and right image lie on the **same row**
- The search for correspondences reduces to a **1D horizontal search**

For a **calibrated** stereo rig (known camera positions and intrinsics), the rectifying image transforms can be computed directly from the calibration — no correspondence estimation between the two images is needed.

Stereo Rectification

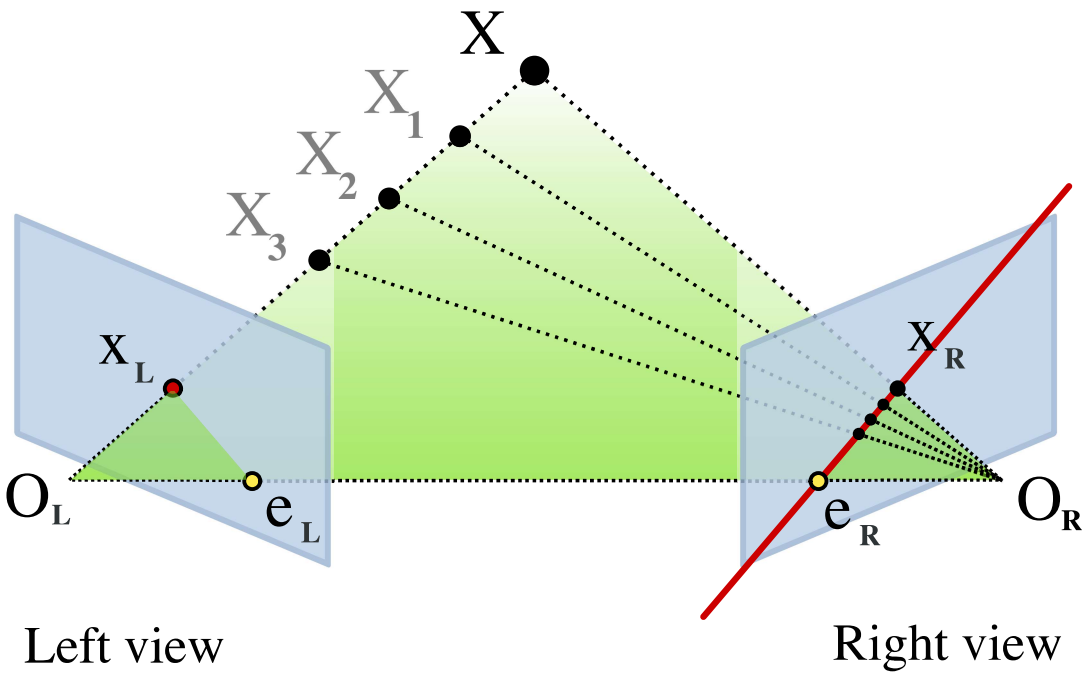


Image source: Arne Nordmann (norro), CC BY-SA 3.0 (https://creativecommons.org/licenses/by-sa/3.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Epipolar_geometry.svg

Disparity

Disparity d is the horizontal pixel shift of the same 3D point between the left and right rectified image:

$$d = u_L - u_R$$

Substituting the projection equations (with baseline b and focal length f):

$$d = u_L - u_R = f\frac{X}{Z} + c_x - \left(f\frac{X-b}{Z} + c_x\right) = f\frac{X-(X-b)}{Z} = \frac{f \cdot b}{Z}$$

Key observation:

- **Nearby objects** → large disparity d
- **Distant objects** → small disparity d
- Objects at infinity → $d = 0$

Depth from Disparity

Rearranging the disparity equation gives the **depth** Z:

$$Z = \frac{f \cdot b}{d}$$

This is the core formula of stereo estimation:

- f: focal length — from calibration
- b: baseline — from the stereo rig geometry
- d: disparity — estimated from the two images

The depth estimation problem reduces to accurately estimating disparity.

Depth from Disparity

Geometric relationship in a stereo camera setup:

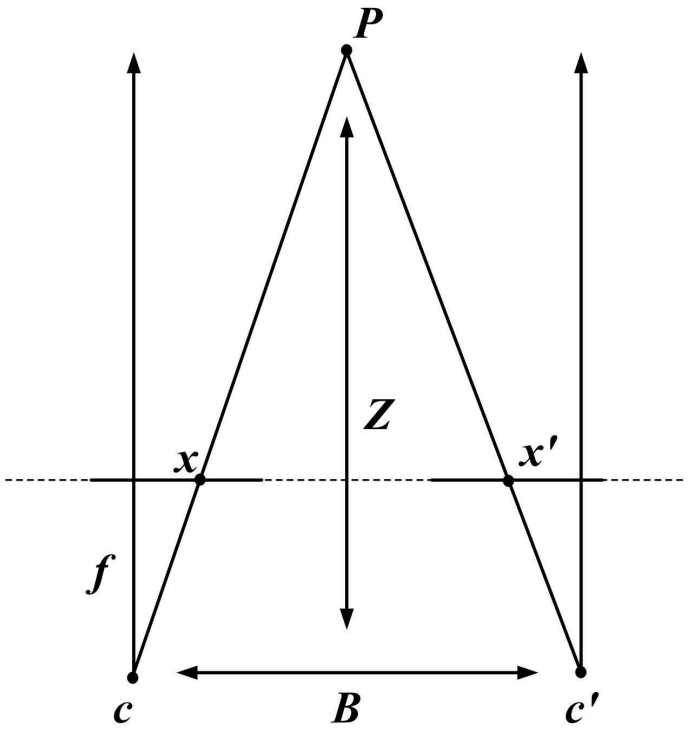


Image source: Daviddengcn, CC BY-SA 3.0 (https://creativecommons.org/licenses/by-sa/3.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Stereo-camera-model.jpg

Stereo Camera

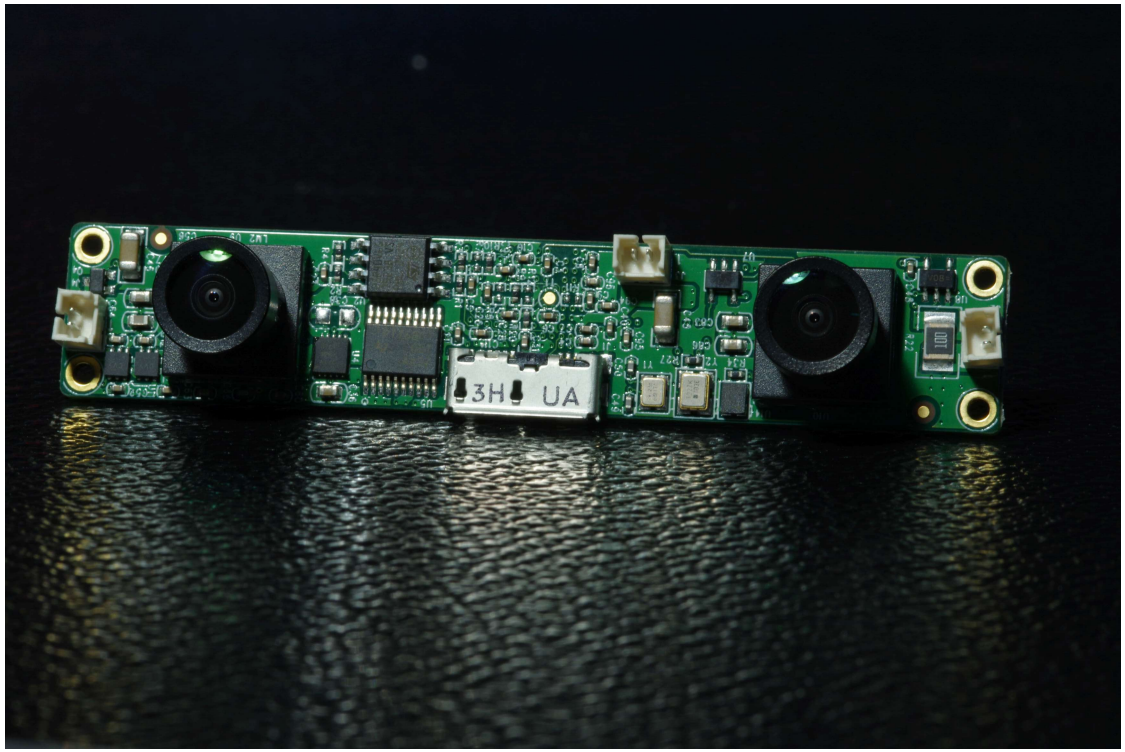


Image source: Timothée Cognard, CC BY-SA 3.0 (https://creativecommons.org/licenses/by-sa/3.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Stereo_camera.JPG

Stereo Camera



Image source: Leif Skandsen, CC BY 2.0 (https://creativecommons.org/licenses/by/2.5), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Stereo_(5460537599).jpg

The Disparity Map

The **disparity map** $D(u, v)$ assigns a disparity value d to each pixel (u, v) of the left image. From the disparity map, a **depth map** can be computed pixelwise:

$$Z(u, v) = \frac{f \cdot b}{D(u, v)}$$

Properties:

- Disparity is **undefined** at occlusions (regions visible in only one image)
- Disparity is **ambiguous** in textureless regions (aperture problem)
- Disparity is often **refined to sub-pixel accuracy** using interpolation or continuous optimization

The Stereo Matching Problem

Given the rectified left image I_L and right image I_R, the task is to find for each pixel (u, v) in I_L the best matching pixel $(u - d, v)$ in I_R.

This is formulated as an **optimization problem**:

$$\hat{D}(u, v) = \arg \min_{d \in [d_{\min}, d_{\max}]} C(u, v, d)$$

where $C(u, v, d)$ is a **matching cost** measuring the dissimilarity between:

- A patch around (u, v) in I_L
- A patch around $(u - d, v)$ in I_R

The range $[d_{\min}, d_{\max}]$ is the **disparity search range** — typically $[0, D_{\max}]$ with $D_{\max} \approx 192$ or 256 pixels.

Matching Cost Functions

Classical matching costs compare **local patches** around corresponding pixels:

Method	Property
Sum of Squared Differences (SSD)	Sensitive to outliers
Sum of Absolute Differences (SAD)	More robust than SSD
Normalized Cross-Correlation (NCC)	Invariant to linear intensity changes of the form $aI + b$ with $a > 0$

Patch-based methods suffer at **occlusions** and **depth discontinuities** because a fixed window crosses multiple depth layers.

Why Deep Learning for Stereo?

Traditional stereo matching relies on **handcrafted** features that struggle with:

- **Textureless regions** (walls, sky, roads)
- **Specular reflections** (wet surfaces, glass)
- **Repetitive patterns** (fences, gratings)
- **Occlusions** at depth boundaries

Why Deep Learning for Stereo?

Deep learning learns **discriminative, context-aware features** directly from data, overcoming these limitations.

Two key paradigms:

1. **Learned cost functions**: replace handcrafted costs with learned similarity measures
2. **End-to-end networks**: jointly learn feature extraction, cost aggregation, and disparity estimation

Feature Extraction for Stereo Matching

In learned stereo, the goal is to extract a **feature vector** $\mathbf{z}(u, v) \in \mathbb{R}^F$ for each pixel (u, v) such that:

- Features of **matching pixels** (same 3D point) are **similar**
- Features of **non-matching pixels** are **dissimilar**

This is a **metric learning** problem: learn an embedding $f_\phi : \mathbb{R}^{H \times W \times C} \to \mathbb{R}^{H \times W \times F}$ where the distance in feature space correlates with 3D correspondence.

The encoder of a **convolutional autoencoder** can serve as an initialization point because it compresses visual content into a compact latent representation. Since reconstruction and correspondence are different objectives, stereo-specific fine-tuning is typically required.

Autoencoder as a Feature Extractor — Recap

Recall from the autoencoder lecture:

- Encoder f_ϕ: maps input $\mathbf{x}$ to latent code $\mathbf{z} = f_\phi(\mathbf{x})$
- The encoder captures the **most informative features** of the input
- A **convolutional autoencoder (CAE)** produces a **dense feature map** $\mathbf{z} \in \mathbb{R}^{H' \times W' \times F}$

For stereo:

- The CAE encoder is applied to **both** the left and right images independently with **shared weights**
- The resulting feature maps $\mathbf{z}_L$ and $\mathbf{z}_R$ are compared against each other

This is the **Siamese encoder** architecture.

Siamese Encoder for Stereo

A **Siamese network** consists of **two branches with shared weights** applied to different inputs — here, the left and right images:

$$\mathbf{z}_L = f_\phi(I_L), \qquad \mathbf{z}_R = f_\phi(I_R)$$

Weight sharing is crucial:

- Ensures that features are **comparable** across images — the same visual pattern maps to the same feature vector
- Reduces the number of parameters
- Enforces consistent representation of the same visual structure in both views

The Siamese encoder is either **pre-trained** (e.g. as part of an autoencoder) or **trained end-to-end** with a stereo-specific loss.

The Cost Volume

After extracting feature maps $\mathbf{z}_L \in \mathbb{R}^{H\times W\times F}$ and $\mathbf{z}_R \in \mathbb{R}^{H\times W\times F}$, a **cost volume** $\mathbf{C} \in \mathbb{R}^{D\times H\times W}$ is built:

$$\mathbf{C}(d,u,v) = -\text{sim}\left(\mathbf{z}_L(u,v),\ \mathbf{z}_R(u-d,v)\right)$$

where $\text{sim}(\cdot,\cdot)$ is a **similarity function —** e.g. the dot product $\langle \mathbf{z}_L, \mathbf{z}_R \rangle$ or cosine similarity. It is written with a minus sign so that **lower values indicate a better feature match** (cost convention). If a distance such as $\|\mathbf{z}_L - \mathbf{z}_R\|_2$ is used instead, no negation is needed.

The cost volume stores a **matching cost** for every pixel (u,v) at every candidate disparity $d \in [0, D_{\max}]$.

Cost Volume Construction: Concatenation

A common approach is the **concatenation cost volume**:

$$\mathbf{C}'(d, u, v) = [\mathbf{z}_L(u, v) \parallel \mathbf{z}_R(u - d, v)]$$

where $\parallel$ denotes feature concatenation. The resulting volume has shape $D \times H \times W \times 2F$.

Advantages:

- Preserves all feature information (no information loss from dot product)
- A following 3D CNN can learn **complex matching functions** from the raw feature pairs

3D Convolutional Cost Aggregation

A **3D convolutional neural network** is applied to the cost volume to perform **context-aware cost aggregation**:

$$\mathbf{C}_{\text{agg}} = g_\theta(\mathbf{C}')$$

The 3D CNN processes the disparity dimension jointly with the spatial dimensions, learning to:

- **Smooth** costs in textureless regions
- **Preserve** disparity discontinuities at object boundaries
- **Fill in** occluded regions using context from neighboring pixels

Disparity Regression

After cost aggregation, a **disparity map** is produced. Two common approaches:

Winner-Takes-All (WTA):

$$\hat{D}(u, v) = \arg\min_{d} \mathbf{C}_{\text{agg}}(d, u, v)$$

Non-differentiable — cannot be trained end-to-end directly.

Disparity Regression

Differentiable soft-argmin (Kendall et al., 2017):

$$\hat{D}(u,v) = \sum_{d=0}^{D_{\max}} d \cdot \frac{\exp\left(-\mathbf{C}_{\text{agg}}(d,u,v)\right)}{\sum_{d'=0}^{D_{\max}} \exp\left(-\mathbf{C}_{\text{agg}}(d',u,v)\right)}$$

This is a probability-weighted average across disparities, where lower matching costs receive larger softmax weights.

The soft-argmin produces **sub-pixel accurate** disparity estimates and is fully differentiable — enabling end-to-end training.

Loss Function for Stereo Networks

For **supervised** stereo estimation, the loss compares the predicted disparity map $\hat{D}$ to the **ground-truth** disparity D^* (for example from synthetic stereo data, structured-light sensors, or depth data projected into the camera pair using calibration).

Smooth L1 loss (Huber loss):

$$\mathcal{L}(\hat{D}, D^*) = \sum_{(u,v)\in\mathcal{V}} \ell_\delta(\hat{D}(u,v) - D^*(u,v))$$

$$\ell_\delta(x) = \begin{cases} \frac{1}{2}x^2 & |x| \le \delta \\ \delta\left(|x| - \frac{\delta}{2}\right) & |x| > \delta \end{cases}$$

where $\mathcal{V}$ is the set of valid (non-occluded) pixels.

Smooth L1 loss (Huber loss)

Huber loss in green, squared loss in blue:

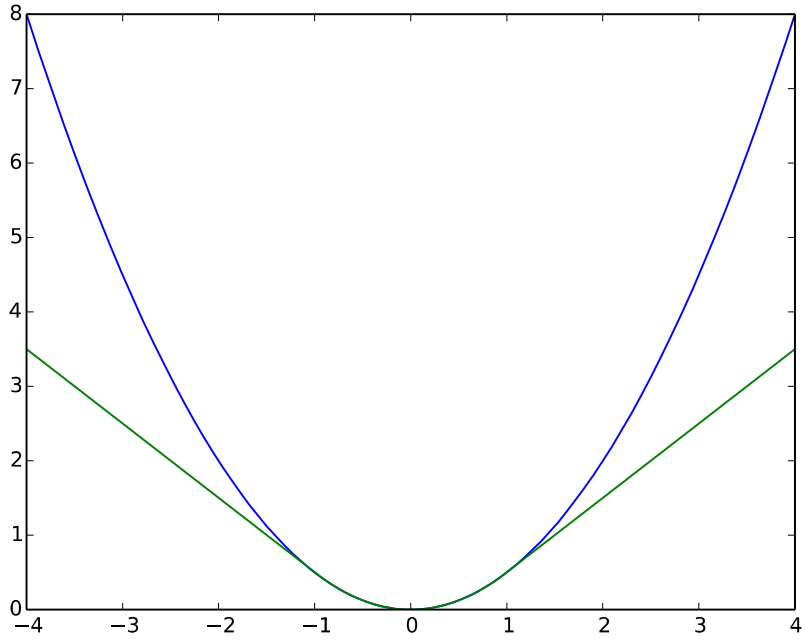


Image source: Qwertyus, CC BY-SA 4.0 (https://creativecommons.org/licenses/by-sa/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Huber_loss.svg

Data Augmentation for Stereo

To improve generalization, **geometry-preserving** augmentations are applied consistently to both left and right images:

- **Color jitter**: brightness, contrast, saturation, hue — applied **identically** to both images
- **Random crop**: crop the same region from both images
- **Random horizontal flip**: only valid if the left/right images are swapped as well and the disparity representation is updated consistently; because this is easy to get wrong, many stereo pipelines avoid it
- **Right-image-only augmentation** (e.g. simulated occlusion patches): applied independently to the right image only

The stereo constraint (epipolar geometry) must be **preserved** by any augmentation applied to both images jointly.

MC-CNN: Learned Matching Cost

MC-CNN (Žbontar & LeCun, 2016) was one of the first deep learning approaches for stereo:

1. Two small CNNs (Siamese) extract features for a pair of patches
2. The feature similarity is used as a **learned matching cost**
3. Cost aggregation and disparity selection remain **classical**

The CNN replaces only the **cost function** — everything else is unchanged.

MC-CNN demonstrated that even a small learned cost function dramatically outperforms handcrafted alternatives, establishing deep features as the foundation of modern stereo.

DispNet: End-to-End Stereo

DispNet (Mayer et al., 2016) was the first **fully end-to-end** convolutional stereo network:

- Each image (I_L, I_R) is processed independently through **shared convolutional layers**
- A **correlation layer** computes matching costs between the two feature maps
- An encoder-decoder architecture produces a disparity map

GC-Net: Geometry and Context Network

GC-Net (Kendall et al., 2017) introduced two key innovations:

1. **Concatenation cost volume**: $\mathbf{C}(d, u, v) = [\mathbf{z}_L(u, v) \parallel \mathbf{z}_R(u - d, v)]$ — richer than a dot-product cost
2. **Differentiable soft-argmin disparity regression**: produces sub-pixel accurate outputs

Architecture:

- Concatenation cost volume $\mathbf{C}'$ (shape $D \times H \times W \times 2F$)
- 3D CNN for context-aware cost aggregation: $\mathbf{C}_{\text{agg}} = g_\theta(\mathbf{C}')$
- Soft-argmin layer for disparity regression

GC-Net established the **cost volume + 3D CNN** paradigm of modern stereo networks.

Autoencoder Pre-Training for Stereo

A convolutional autoencoder can be used to **pre-train** the stereo feature extractor:

1. Train a CAE on a large collection of **unlabeled images**:

$$\mathcal{L}_{\text{AE}} = \|I - g_\psi(f_\phi(I))\|_2^2$$

2. Extract the encoder f_ϕ and use it as the **Siamese feature extractor**
3. Fine-tune the complete stereo network end-to-end with the stereo loss

Benefits:

- The encoder has learned **generalizable visual features** before seeing any stereo supervision
- Pre-training reduces the amount of **labeled stereo data** required

Denoising Autoencoder for Robust Features

A **denoising autoencoder (DAE)** pre-trained on images corrupted with Gaussian noise or occlusion patches can yield robust features for stereo matching:

- The DAE encoder learns features **invariant to local noise** — beneficial for matching in noisy or low-texture regions
- By training with occluded patches, the encoder learns to **tolerate missing information** — directly relevant to occlusion in stereo

$$(\phi, \psi) = \arg\min_{\phi,\psi} \mathbb{E}\left[\|I - g_\psi(f_\phi(I_{\text{noisy}}))\|_2^2\right]$$

where I is the clean image and I_{noisy} is the corrupted input.

The DAE pre-training acts as a **self-supervised regularizer**, improving stereo performance especially in difficult regions.

End-to-End Stereo: Complete Pipeline

The complete deep stereo estimation pipeline:

$$I_L, I_R \xrightarrow{f_\phi \text{ (Siamese encoder)}} \mathbf{z}_L, \mathbf{z}_R \xrightarrow{\text{cost volume}} \mathbf{C} \xrightarrow{g_\theta \text{ (3D CNN)}} \mathbf{C}_{\text{agg}} \xrightarrow{\text{soft-argmin}} \hat{D} \xrightarrow{\text{depth conversion}} \hat{Z}$$

with $\hat{Z}(u,v) = \dfrac{fb}{\hat{D}(u,v)}$ for valid disparities; in practice, pixels at occlusions, mismatches, or with non-positive predicted disparity are treated as invalid or unreliable.

End-to-End Stereo: Complete Pipeline

The **trainable pipeline up to disparity regression** is differentiable and can be optimized end-to-end by minimizing the disparity loss $\mathcal{L}(\hat{D}, D^*)$; the depth conversion is then an analytic post-processing step for valid disparities.

Benefits of end-to-end training:

- The encoder learns features **optimized for stereo matching**, not just image reconstruction
- The aggregation network learns to handle occlusions and discontinuities

Applications of Stereo Estimation

Stereo depth estimation is a key component in many real-world systems:

Application	Role of Stereo Estimation
Autonomous driving	Obstacle distance, free-space detection
Robotics	Object grasping, collision avoidance
AR/VR	Scene reconstruction for augmentation
Industrial inspection	Dimensional measurement, defect detection
Medical imaging	Endoscopic 3D reconstruction
UAV navigation	Terrain following, landing

Challenges in Stereo Estimation

Despite impressive progress, stereo estimation still faces open challenges:

- **Occlusions**: pixels visible in only one camera have no valid correspondence
- **Textureless regions**: insufficient information to disambiguate disparities (aperture problem)
- **Reflections and transparency**: violate the Lambertian surface assumption
- **Large disparity range**: high-resolution cameras require searching hundreds of pixels
- **Domain gap**: networks trained on synthetic data may fail on real-world scenes
- **Real-time constraints**: 3D CNN cost aggregation is computationally expensive

These challenges drive ongoing research in **efficient**, **robust**, and **generalizable** stereo networks.

Summary: Stereo Geometry

Key geometric results:

- **Disparity** $d = u_L - u_R$ measures the pixel shift between rectified stereo images
- **Depth** $Z = \frac{f \cdot b}{d}$ — depth is inversely proportional to disparity
- **Epipolar constraint**: corresponding pixels lie on the same row after rectification — reducing stereo matching to a 1D search
- For a small fixed disparity error Δd, first-order error propagation from $Z = \frac{fb}{d}$ gives $|\Delta Z| \approx \left|\frac{\partial Z}{\partial d}\right| |\Delta d| = \frac{Z^2}{fb} |\Delta d|$ — long baselines improve far-range accuracy

Summary: Deep Stereo Estimation

- **Siamese encoder** f_ϕ extracts comparable features from both images (shared weights)
- **Cost volume** $\mathbf{C}(d, u, v)$ stores matching costs for all pixels and all disparity candidates
- **3D CNN aggregation** propagates context across space and disparity, replacing hand-tuned aggregation
- **Soft-argmin regression** produces sub-pixel accurate, differentiable disparity estimates
- **Autoencoder pre-training** initializes the encoder with robust visual features before stereo fine-tuning

Stereo Estimation

Recap!

- 📐 **Geometry**: disparity $d = u_L - u_R$ and depth $Z = \frac{f \cdot b}{d}$ — depth is inversely proportional to disparity; epipolar geometry reduces matching to a 1D search along rectified rows
- 🔍 **Matching**: the stereo problem is solved by finding corresponding pixels; classical methods use handcrafted costs with aggregation; deep methods learn costs and aggregation end-to-end
- 🧠 **Siamese encoder**: shared-weight convolutional encoder extracts comparable feature maps from both images; a cost volume stores all pairwise feature similarities across disparities